M4 — New room API surface (no UI yet)

Phase: 2 — Routes **Status:** DONE (2026-05-26) **Blocked by:** M3 **Blocks:** M5, M6, M7, M10 **Estimated effort:** 2 days · **Actual:** 0.5 day (the underlying ControlRoom methods landed in M2; M4 was a thin HTTP layer)

1. Purpose

Add eight HTTP routes that let an external client (the charge-nurse dashboard in M5, the wizard's room-mode branch in M6, future automation scripts) drive the multi-patient layer. No UI in this module — that's M5's job. The routes are the contract; M5 consumes them.

The routes split into three families:

Family	Routes
Lifecycle	`POST /api/room/start`, `GET /api/room/state`, `POST /api/room/end`
Synchronized control	`POST /api/room/freeze_all`, `POST /api/room/resume_all`, `POST /api/room/scene_broadcast`
Per-encounter	`POST /api/encounter/{id}/scene`, `POST /api/encounter/{id}/assign_students`

2. Structure

Files touched:

• `portal/server.py` — imports `control_room`; appends an "M4" section

at the file's end with the 8 route handlers and four helper functions (`_encounter_summary`, `_room_summary`, `_require_active_room`, `_require_encounter`, `_apply_scene`).

No new files — everything lives in `server.py` alongside the v6 routes. M7 will extract scene logic into `portal/scenes.py` and replace `_apply_scene` with the templated palette.

3. Uses

• M5 (charge-nurse dashboard) — `GET /api/room/state` is polled every

2 s; the freeze/resume buttons call the `/api/room/freeze_all` and `/api/room/resume_all` endpoints.

• M6 (wizard step-0 toggle) — the room-mode branch POSTs to

`/api/room/start`.

• M7 (scenes engine) — replaces `_apply_scene` here with a templated

per-kind palette.

• M14/M15 (cohort debrief) — reads room/encounter records persisted

by `/api/room/end`.

• M16 (WebSocket transport) — keeps the persistence path through

these routes; swaps the *delivery* of the state change from HTTP-poll to WebSocket push.

4. Functions (exported API surface)

HTTP routes

Method	Path	Purpose	Body	Returns	
POST	`/api/room/start`	Create a ControlRoom with N encounters	`{label, encounters: [{scenario_name, persona_id, ehr_id, chart_mode, ...}], haiku_rate_cap?, voice_char_cap?}`	`{ok, room_id, room_code, encounters: [{encounter_id, join_code, scenario_name}]}`	
GET	`/api/room/state`	Aggregate room + per-encounter summary	—	room summary dict (see below)	
POST	`/api/room/freeze_all`	Set every encounter to paused; room.status = frozen	—	`{ok, status, encounter_count}`	
POST	`/api/room/resume_all`	Inverse	—	`{ok, status, encounter_count}`	
POST	`/api/room/end`	End each encounter, clear singleton	—	`{ok, room_id, encounter_count}`	
POST	`/api/room/scene_broadcast`	Inject a scene into many encounters	`{scene: {...}, targets: "all" \`	[encounter_id, ...]}`	`{ok, fired, results}`
POST	`/api/encounter/{id}/scene`	Inject a scene into one encounter	`{scene: {...}}`	`{ok, event_id, encounter_id}`	
POST	`/api/encounter/{id}/assign_students`	Replace roster on encounter	`{student_ids: [...]}`	`{ok, encounter_id, assigned_student_ids}`	

`/api/room/state` response shape

```
{
  "room_id": "...", "room_code": "ABCD34", "label": "Morning Shift",
  "status": "active",  // active | frozen | ended
  "created_at": 1700.0, "ended_at": null,
  "haiku_rate_cap": null, "voice_char_cap": null,
  "encounters": [
    {
      "encounter_id": "...", "join_code": "ALPHA1",
      "label": "Bed 1 — Diaz", "scenario_name": "...",
      "patient_persona_id": "P-001", "state": "running",
      "chart_mode": "shared", "ehr_id": "helix",
```

```
        "chat_stations": 2, "ehr_stations": 1, "device_stations": 0,
        "chart_event_count": 7,
        "assigned_student_ids": ["s_xxx", "s_yyy"],
        "last_event_ts": 1700.0
      },
      ...
    ],
    "students": [
      {"student_id": "...", "display_name": "...",
       "assigned_encounter_id": "...", "registered_at": 1700.0,
       "last_seen": 1700.0}
    ]
  }
```

Helpers

Symbol	Purpose
`_encounter_summary(enc)`	Per-encounter dict for the poll body. Cheap (no fold).
`_room_summary(room)`	Aggregate poll body.
`_require_active_room()`	404 if no room is active.
`_require_encounter(id)`	404 if encounter id is not in the active room.
`_apply_scene(enc, scene, by=)`	Minimal scene applicator: writes one `instructor.trigger` chart_event scoped to the encounter. **M7 replaces this** with the full scenes palette.

5. Limitations

• **Scenes are minimal.** `_apply_scene` writes a single

`instructor.trigger` chart_event carrying the scene dict verbatim. M7 expands this into per-kind handlers (`vitals.drop`, `lab.result`, `family.arrives`, `code.blue`, etc.) that emit multiple typed events. The wire format on `/api/*/scene` is forward-compatible — clients send `{"kind": "...", "params": {...}}` today and M7 just adds richer behavior on the server side.

• **No WebSocket push yet.** Freeze/resume/scene state changes

propagate via HTTP poll. Stations pick up the change on the next /api/room/state or per-station poll. M16 swaps the transport.

• **Single-instructor model preserved.** There is exactly one

`_active_room` at a time. A `POST /api/room/start` while another room is active ends the old room cleanly first.

• **Roster persistence is in-memory.** `_apply_scene` writes

durably to `chart_event`, but the `student` table writes land in M8 (the M4 assign-students route mutates the in-memory ControlRoom state only). Restart-survival of rosters is M8's contract.

• **Auth is per-route operator-only.** Every route requires

`auth.require_vault`. The observer (read-only) seat lands in M18.

• **End does not build cohort debrief.** `/api/room/end` clears the

singleton; the debrief aggregation is M14's responsibility.

6. Test status

Test	Asserts	Status	Last run
`test_api_room_start_creates_room_with_n_encounters`	Distinct encounter ids + join codes; state reflects room.	PASS	2026-05-26
`test_api_room_freeze_all_pauses_each_encounter`	room.status='frozen'; every encounter.state='paused'.	PASS	2026-05-26
`test_api_room_resume_all_restores_state`	running → paused → running round-trip.	PASS	2026-05-26
`test_api_room_state_returns_per_encounter_summary`	Aggregate fields + the 14 per-encounter fields.	PASS	2026-05-26
`test_api_scene_broadcast_writes_chart_event_per_target`	One chart_event per target encounter; targeted broadcast doesn't bleed.	PASS	2026-05-26
`test_api_encounter_scene_targets_one_encounter`	Single-encounter scene leaves siblings untouched.	PASS	2026-05-26
`test_api_room_end_clears_singleton_and_404s_subsequent_state`	End → 404 on state.	PASS	2026-05-26
`test_api_encounter_assign_students_replaces_roster`	Roster replace + sweep of previously-bound students.	PASS	2026-05-26
`test_api_room_routes_404_when_no_room`	Aggregate routes 404 without an active room.	PASS	2026-05-26
`test_api_encounter_routes_404_on_unknown_encounter`	Per-encounter routes 404 on unknown id.	PASS	2026-05-26

All 10 in `tests/v7/test_room_api.py` — **PASS** under the v6 venv.

7. Change list

Date	Author	Change	Files
2026-05-26	claude-code	8 new routes + 5 helpers; `test_room_api.py` with 10 cases. Full v7 suite: 134 passed (up from 124), same 6 pre-existing env-flaky failures as v6 baseline. Zero regressions.	`portal/server.py`, `tests/v7/test_room_api.py`

8. Open questions / known issues

• The `_apply_scene` station-id fallback (`f"instructor:{by}"`) writes

a synthetic station id when no EHR station has joined the encounter yet. The `chart_event.ehr_station_id` column is NOT NULL, so we satisfy that constraint, but the synthetic id won't match any registered station row. The `fold()` reader tolerates this (it doesn't validate station ids). M7

should keep the same fallback.

• The `/api/room/start` body shape uses `persona_id` AND

`patient_persona_id` interchangeably (the new column is `patient_persona_id`, but legacy callers may send `persona_id`). Picked the patient_persona_id form for the new field, fall back to `persona_id` for compat. M6 (the wizard) standardizes on `patient_persona_id` everywhere.

• `/api/encounter/{id}/assign_students` replaces the roster

wholesale; partial-edit semantics (add/remove individual students) are deliberately out of scope. M5's dashboard adds via the full-replace pattern.

• No rate-limit on `/api/room/state` polling. Charge-nurse dashboard

polls every 2 s; an automated client could hammer this. M19 capacity hardening should add a basic per-IP throttle.